

CREATE: A Closed-Loop Reward Editing Agent with Training Evidence for Reinforcement Learning

Anonymous Authors
Anonymous Submission

I. INTRODUCTION

CREATE is a closed-loop reward editing agent that treats policy training as diagnostic evidence. Component-level statistics enable semantic failure diagnosis; hierarchical editing constrains interventions for single-trial attribution; and intervention memory preserves the evidence–action–outcome chain. On LunarLander-v3, CREATE solves all five lineages under a ten-evaluation budget, versus zero for budget-matched independent generation and stateless revision. Ablations confirm enriched evidence and bounded editing as coupled requirements.

Reinforcement learning (RL) depends critically on reward functions that express task intent while providing informative feedback for policy optimization. A semantically plausible reward can nevertheless fail through sparse signals, conflicting components, exploitable proxies, or inappropriate scales, degrading both learning efficiency and final behavior quality [1]–[4].

Reward engineering is therefore iterative. Each completed training run produces rich diagnostic signals—episode length, termination patterns, component activation, and training dynamics—that can reveal how a reward program shapes the learned policy. Converting these heterogeneous records into a precise reward modification, however, still requires substantial human analysis.

Code-capable large language models (LLMs) offer a route to automation. Methods such as Language to Rewards, Text2Reward, and Eureka generate and iteratively refine reward programs from task descriptions and training feedback [5]–[11]. In parallel, LLMs are increasingly organized as agents that observe, reason, and act across trials [12]–[14]. These developments motivate an agent-oriented view of reward engineering in which policy training supplies observations and reward-code revision constitutes an external action.

This paper explores that view through CREATE, a *Closed-Loop Reward Editing Agent with Training Evidence*. CREATE places a reward-editing agent above an inner RL policy learner. After each policy-training run, a tool-using subagent compresses native-task outcomes, reward-component statistics, and training dynamics into a structured evidence summary. The reward-editing agent diagnoses one principal semantic failure, selects an edit level (L1/L2/L3), and applies a semantically localized repair. Intervention memory retains the evidence–action–outcome chain across rounds, while a separate best archive protects the strongest solution.

Experiments on LunarLander-v3 compare CREATE against budget-matched independent generation and stateless iterative revision, with two mechanism ablations isolating evidence enrichment and hierarchical semantic editing. BipedalWalker-

v3 provides a secondary cross-task validation. Under a ten-evaluation budget, the experiments examine whether structured observation, persistent state, and bounded reward actions can convert failed policy training into reliable reward repair.

II. RELATED WORK

A. Reward Engineering for Reinforcement Learning

Reward shaping augments task objectives with intermediate signals that improve credit assignment while preserving the optimal policy under certain transformations [2]. Inverse reward design treats a specified reward as imperfect evidence of designer intent [3]; related approaches infer objectives from demonstrations, preferences, or intrinsic motivation. Evolutionary methods and meta-gradients treat reward parameters as objects of optimization, providing the foundation for language-model-based reward engineering.

B. LLM-Based Reward Generation and Refinement

LLMs can map task descriptions to executable reward code. Language to Rewards and Text2Reward generate reward programs for robotic and locomotion tasks [5], [6]; Eureka combines reward-code generation with evolutionary optimization and component-level feedback [7]; subsequent work incorporates dynamic feedback, human input, and structured reasoning [8]–[10]. Diagnostic-driven refinement treats reward failure as a debugging problem and uses training diagnostics for targeted revision [11]. Collectively, these methods show that training feedback supports reward revision beyond one-shot generation.

C. LLM Agents and Agentic Reward Engineering

Language-agent research organizes LLMs to maintain state and act in external processes. ReAct couples reasoning with tool-mediated actions; Reflexion stores verbal feedback as episodic experience [12], [13]. These mechanisms are entering reward design: RF-Agent formulates reward construction as sequential decision making [15], and STRIDE applies agentic engineering to humanoid locomotion [16]. CREATE follows this direction but centers the interaction on instrumented policy training: a diagnostic subagent converts training records into observations, and the reward-editing agent commits to a semantically localized edit measured by the next complete training run.

III. CREATE

CREATE comprises an outer reward-editing agent and an underlying RL agent. The RL agent interacts with the environment and learns a policy under the current generated reward, whereas CREATE treats each completed policy-training

and native-evaluation run as one reward-design round. As shown in Fig. 1, tool-assisted evidence processing organizes the resulting records before the reward-editing agent diagnoses the current reward and executes a validated edit.

The resulting observe–reflect–act loop is written as

$$\begin{aligned} O_t &= \Phi_{\mathcal{T}}(\mathcal{D}_t), \\ (H_t, A_t) &= \Psi(R_t, O_t, M_t, B_t), \\ R_{t+1} &= \mathcal{U}(R_t, A_t), \end{aligned} \quad (1)$$

where the training record $\mathcal{D}_t$ is retrieved and compressed into observation O_t through evidence tools. This processing stage organizes information but does not select a reward edit. The reward-editing agent combines O_t with the current reward R_t , intervention memory M_t , and best archive B_t to produce reflection H_t , reward action A_t , and the next executable reward program R_{t+1} .

A. Task Understanding and Reward Initialization

Given the task description, environment interface, and reward-code constraints, CREATE identifies the principal behavior semantics and instantiates them as named reward components. The initial reward is

$$R_0(s, a, s') = \sum_{i=1}^{m_0} w_i^{(0)} r_i^{(0)}(s, a, s'), \quad (2)$$

where each component represents a behavior semantic such as task progress, stability, target events, or action cost. Named components are logged independently during training, enabling component-level diagnosis in later rounds. Generated code is checked for interface and execution validity before policy training; prompts and validation rules are provided in the supplementary material.

B. Training-Evidence-Driven Observe–Reflect–Act

At iteration t , the underlying RL agent optimizes the generated reward, while CREATE evaluates the learned policy with the environment-native objective:

$$\begin{aligned} \pi_t &= \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{k=0}^{H-1} \gamma^k R_t(s_k, a_k, s_{k+1}) \right], \\ J_t &= \frac{1}{N} \sum_{n=1}^N \sum_{k=0}^{H_n-1} r_{\text{env}}(s_k^{(n)}, a_k^{(n)}, s_{k+1}^{(n)}). \end{aligned} \quad (3)$$

Thus, the generated reward shapes policy learning, whereas the native objective measures task completion. Each run produces episode outcomes, termination patterns, component activation and scale, temporal dynamics, and differences from the preceding reward version. An auxiliary tool-using subagent retrieves and compresses these records into a structured evidence summary. It reports what occurred during training but does not recommend coefficient changes, select an intervention level, or modify reward code.

Component statistics serve as diagnostic cues rather than fixed editing rules. Persistent inactivity may indicate an unreachable gate or sparse formulation; excessive activation and magnitude may reveal a dominant proxy; frequent activation with weak contribution may indicate insufficient scale, saturation, or an ineffective mathematical form. The reward-editing

agent combines these cues with policy outcomes, reward source code, and prior interventions to identify one principal reward semantic for the next edit:

$$A_t = (q_t, \ell_t, \Delta_t), \quad \text{supp}(\Delta_t) \subseteq \mathcal{C}(q_t), \quad (4)$$

where q_t denotes the selected semantic, ℓ_t the edit level, and $\mathcal{C}(q_t)$ the components and parameters implementing that semantic. An ordinary action may add or remove a component, adjust its weight or gate, or replace its mathematical form. Related components may be adjusted jointly when they express the same semantic, but independent design intents are not mixed within one ordinary revision. L1 calibrates coefficients, thresholds, or gates; L2 adds, removes, or refactors components while preserving the target semantic; and L3 replans the reward structure after repeated local failure.

C. Intervention Memory and Best-Reward Selection

After each round, CREATE records the observation, diagnosed semantic, reward action, expected change, and subsequent outcome in persistent intervention memory. This history supports comparison across rounds and discourages repeated unsuccessful edits. The best archive is maintained separately from the active lineage to prevent later regressions from overwriting a previously discovered solution. After at most K reward evaluations, CREATE returns

$$t^* = \arg \max_{0 \leq t < K} J_t, \quad R^* = R_{t^*}. \quad (5)$$

The selected reward–policy pair is subsequently evaluated on independent episode seeds to assess stability.

D. Summary

Together, the three-stage architecture—evidence processing (Eq. 1), semantic diagnosis and bounded editing (Eq. 4), and intervention memory with best-reward selection (Eq. 5)—forms a closed diagnostic loop. Each completed policy-training run produces component-level evidence that reveals *which* semantic failed and *how*; the editor commits to a targeted intervention on that semantic alone; and the next training run measures whether the diagnosis was correct. Intervention memory preserves this chain across rounds, while the best archive guards against regression. The experiments in Section IV and V test whether this diagnostic architecture can convert failed policy training into reliable reward repair under a fixed evaluation budget.

IV. EXPERIMENTAL PROCEDURE

This section details the experimental protocol that implements the CREATE framework described in Section III. We describe the setup, the five conditions under comparison, and the training and evaluation pipeline. Results are deferred to Section V.

A. Experimental Setup and Configuration

Experiments are conducted on two Gymnasium environments. LunarLander-v3 serves as the primary benchmark for baseline comparison and mechanism ablation; BipedalWalker-v3 provides a secondary validation on continuous-action locomotion. Every reward program trains a freshly initialized PPO policy through Stable-Baselines3. LunarLander uses 10^6 environment

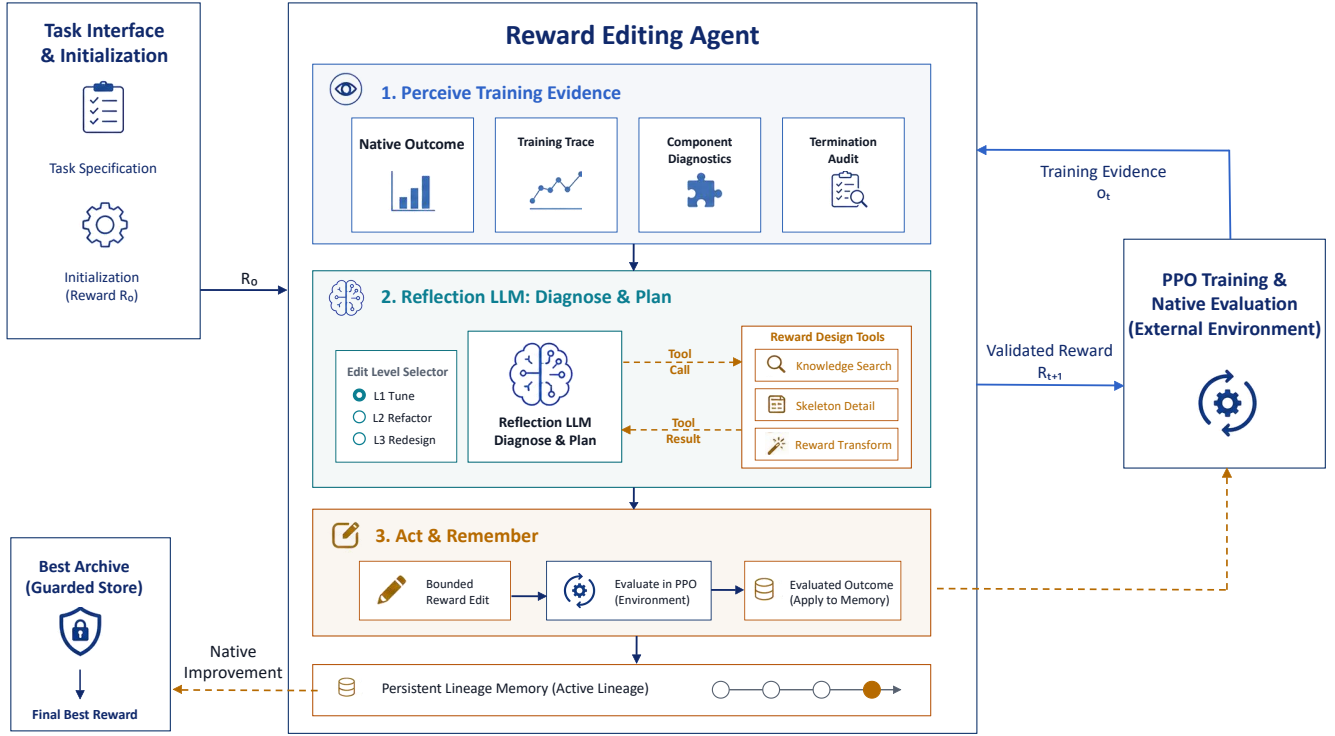


Fig. 1. Overview of CREATE. The underlying RL agent learns under the current generated reward. Tool-assisted evidence processing retrieves and compresses relevant training and lineage records, while the reward-editing agent remains responsible for failure diagnosis, semantic localization, intervention-level selection, and reward modification. Persistent memory records intervention outcomes, and the best archive retains the reward program with the highest native-task performance.

steps per candidate and BipedalWalker uses 5×10^6 . DeepSeek V4 Pro serves as the LLM backbone for all LunarLander conditions. Full prompts, model settings, hyperparameters, per-seed traces, and detailed environment configurations are provided in the supplementary material.

B. Condition Definitions

Five conditions are compared on LunarLander-v3 under a shared LLM backbone, environment description, reward interface, code checks, and PPO budget. Each condition contains five independent reward-search lineages and at most ten complete reward evaluations, so that every condition operates under the same total policy-training cost. All four iterative conditions (Stateless revision, CREATE without evidence enrichment, CREATE without hierarchical editing, and CREATE) share a common initial reward produced by CREATE’s task-understanding phase (Section III-A); the single-shot evaluation of this reward yields a mean fitness of -2.21 ± 73.38 (0/5 solved), confirming that an initially plausible reward is not reliably learnable without subsequent refinement.

Independent-10 generates ten unrelated initial reward candidates, each from an independent invocation of the reward-initialization phase, and evaluates each with a separate policy-training run. The best candidate is selected post hoc. This condition spends the same maximum number of evaluations as CREATE but without cross-round information flow.

Stateless iterative revision retains the sequential LLM invocation of CREATE but removes the evidence organizer,

intervention memory, cross-round component comparison, and hierarchical semantic editing. The LLM receives only basic current-round feedback (native fitness, episode length, termination statistics, component episode sums) and may modify any number of reward components simultaneously, with no constraint on edit scope or semantic localization. This condition serves as a comparison point that isolates the contribution of structured evidence, persistent memory, and bounded editing.

CREATE without evidence enrichment retains the full loop, memory, best archive, and hierarchical editing, but removes the tool-using evidence organizer: the editor receives native fitness, episode length, termination statistics, and component episode sums, but lacks activation rates, magnitude shares, temporal trends, and cross-round summaries.

CREATE without hierarchical editing retains enriched evidence, memory, and the best archive, but removes the semantic-localization constraint (Eq. 4), allowing unconstrained multi-component rewrites.

CREATE runs the complete architecture described in Section III.

C. Training and Evaluation Protocol

Each reward-design round consists of a complete PPO training run followed by native evaluation. The policy is optimized with the generated reward R_t , whereas candidate quality is measured only by the untouched environment reward r_{env} (Eq. 3). We define *search fitness* as the mean undiscounted

TABLE I
LUNARLANDER-V3 REWARD SEARCH OVER FIVE LINEAGES. VALUES ARE
MEAN $\pm$ SAMPLE STANDARD DEVIATION OF LINEAGE-WISE BEST SEARCH
FITNESS.

Condition	Budget	Best fitness	Solved
Independent-10	10	-0.74 ± 114.40	0/5
Stateless revision	10	44.0 ± 65.9	0/5
CREATE w/o evidence enrichment	10	134.97 ± 148.43	2/5
CREATE w/o hierarchical editing	10	114.21 ± 46.31	0/5
CREATE	10	228.98 ± 16.54	5/5

native episodic return over 20 fixed evaluation episodes and *best search fitness* as the largest value retained within a lineage. After reward selection, the returned policy is evaluated over 100 unseen episodes; this is reported as *test fitness*. The task-solving thresholds are 200 for LunarLander and 300 for BipedalWalker.

Between rounds, the evidence organizer (Eq. 1, Φ_T) compresses training records into a structured summary. The reward-editing agent then selects one principal semantic, chooses an edit level (L1/L2/L3), and applies a bounded modification (Eq. 4). The modified program is validated before the next round. The best archive is updated only when a new candidate exceeds the current incumbent in native search fitness (Eq. 5). On BipedalWalker-v3, only the complete CREATE workflow is evaluated; no baseline or ablation comparison is conducted on that task.

V. RESULTS AND ANALYSIS

A. Reward Search Under a Matched Training Budget

Search breadth and iterative rewriting alone do not produce reliable reward repair. Table I tests whether the diagnostic architecture of CREATE—rather than candidate count or repeated LLM invocation—accounts for performance. All four iterative conditions start from a common initial reward (single-shot fitness -2.21 ± 73.38 , 0/5 solved), ensuring that subsequent differences reflect the repair process rather than initialization. Independent-10 fails across all five lineages despite evaluating ten unrelated reward candidates, directly ruling out the alternative explanation that candidate count alone drives improvement. Ten independent guesses, each made without access to training dynamics, remain guesses: failed policy training contains diagnostic information that a fresh initialization cannot recover.

Iterative rewriting without persistent evidence produces non-monotonic progress. Stateless revision raises the mean lineage-wise best fitness to 44.0 from the shared initial reward but solves no seed. In three of five lineages, the initial reward is already the best version, and the final revision is worse than the stored best in all five. Each revision operates on the current reward text alone—without memory of which prior edits succeeded or failed, and without structured evidence to distinguish a poorly calibrated component from a fundamentally wrong semantic. The process therefore accumulates rewrites without accumulating progress. CREATE instead solves all five lineages at rounds 8, 10, 2, 4, and 3, requiring 5.4 complete reward evaluations on average to first cross the threshold. Its selected policies obtain 231.63 ± 12.23 test fitness, close to their search-time performance.

Fig. 2 shows that independent generation improves only when an unrelated sample happens to be better, while stateless

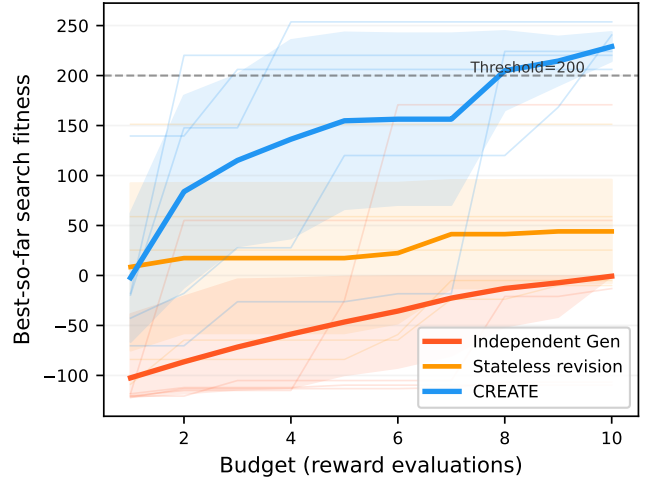


Fig. 2. Best-so-far LunarLander search fitness under a matched policy-training budget. Each curve averages five lineages after retaining the incumbent within each lineage; the dashed line denotes the task-solving threshold.

revision plateaus despite repeated rewriting. CREATE progressively raises its incumbent and solves every lineage. The key distinction is whether a completed training run becomes persistent diagnostic evidence rather than a binary outcome. Held-out test fitness (Fig. 3) confirms that CREATE attains the highest mean with the lowest dispersion.

B. Mechanism Ablations

The previous section establishes that the closed loop works. We now isolate *why* it works by testing two mechanism hypotheses derived from the design choices in Section III.

Enriched evidence enables diagnosis. The enriched evidence stream (activation rates, magnitude shares, temporal trends, cross-round summaries) enables the editor to distinguish between mechanistically distinct failure modes. Without this information, the editor operates on coarse aggregate statistics that conflate different root causes—an inactive component could be stuck behind an unreachable gate, too weak to compete, or behaviorally irrelevant, and a scalar episode sum cannot tell them apart.

Removing the tool-using evidence organizer leaves the editor with native fitness, episode length, termination statistics, and component episode sums, but removes activation rates, magnitude shares, temporal trends, and structured cross-round summaries. This variant solves two seeds but exhibits very high dispersion (134.97 ± 148.43 ; Table I), including one lineage that remains negative throughout all ten rounds. Basic component values can therefore support occasional repair, but cannot reliably answer the diagnostic questions that matter. The evidence supports this link: *what* training evidence is provided determines *which* diagnoses are possible.

Hierarchical editing enables single-trial attribution. The semantic-localization constraint (Eq. 4) ensures that each expensive retraining run serves as a controlled test of a single intervention hypothesis. Without this constraint, multi-component edits prevent the system from attributing a subsequent performance change to any specific modification—the credit assignment problem moves from RL training to reward design itself.

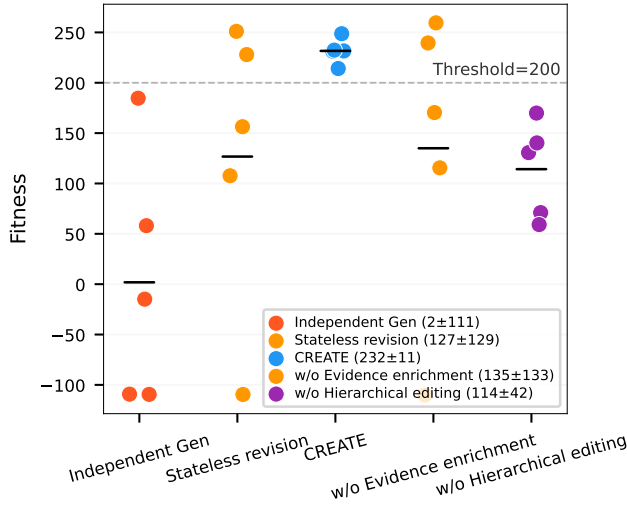


Fig. 3. Per-seed held-out test fitness and mechanism ablation results on LunarLander. Horizontal bars mark group means; the dashed line denotes the task-solving threshold (200).

Removing hierarchical semantic editing retains enriched evidence, memory, and the best archive, but allows unconstrained multi-component rewrites. This variant reaches moderate fitness in several seeds yet solves none (114.21 ± 46.31), and one lineage collapses to approximately -5511 in two consecutive rounds. The failure is not a lack of information: the same enriched evidence is available, but when several unrelated components change together, one subsequent training result cannot identify which intervention should be retained or reverted. Each retraining run (10^6 environment steps) becomes an uncontrolled multivariate experiment rather than a focused hypothesis test. The evidence supports this constraint: bounded action is necessary for single-trial attribution.

Fig. 3 shows that enriched evidence and bounded editing describe coupled requirements. Evidence-poor repair can occasionally succeed but is strongly initialization-dependent. Unconstrained editing has lower dispersion only because all five lineages remain below the threshold. Full CREATE combines a 5/5 solution rate with a substantially tighter distribution ($\sigma = 16.54$ vs. 148.43 and 46.31). Neither enriched observation nor bounded action alone suffices; together they enable the diagnostic closed loop.

C. How Evidence Becomes an Edit

Aggregate results establish *that* enriched evidence and hierarchical editing matter. We now examine *how* component evidence is translated into edits. Fig. 4 shows two representative CREATE lineages; activation-rate and magnitude-share heatmaps are in the supplementary material.

Collapse and recovery (seed 0). Rounds 1–6 explore coefficient scaling, sparse-to-dense conversion, and mathematical-form changes without solving the task. Round 7 replaces a state-based proximity signal with differential progress shaping and collapses to approximately -389 . The evidence organizer reveals the mechanism: the new signal is near zero when the lander is stationary and becomes negative under disturbance, actively penalizing stability. The intervention memory retains the pre-collapse reward in the best archive. The editor uses

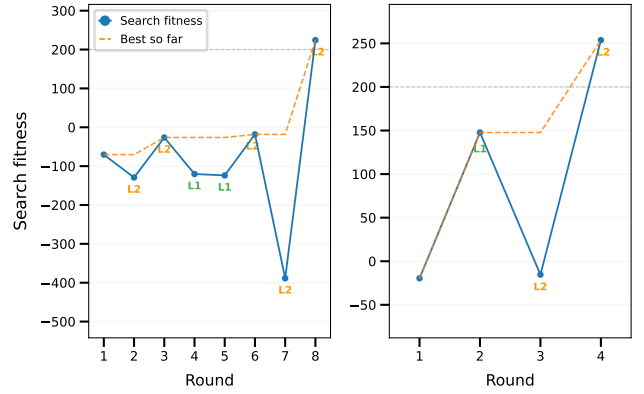


Fig. 4. Two representative CREATE repair trajectories on LunarLander. Solid markers show search fitness per round; the dashed line tracks the best-so-far incumbent. L1/L2 labels mark the edit level at each intervention.

TABLE II
BIPEDALWALKER-V3 RESULTS FOR COMPLETE CREATE ONLY. v DENOTES THE EVALUATED REWARD VERSION.

Seed	Initial fitness	First $v \geq 300$	Best fitness
0	270.7	v_2	320.0
1	280.5	v_2	313.7
2	272.1	v_2	307.9
3	290.0	v_2	311.1
4	103.0	v_3	304.9
Mean	243.26 ± 70.45	2.2 versions	311.54 ± 5.17

this evidence to diagnose that the signal lacks a stable reference frame and applies an L2 intervention in round 8: restore a state-based formulation with an altitude factor that suppresses reward exploitation. Fitness rises to 224. The best archive guards this solution when the next exploratory reward falls to approximately -612 , demonstrating evidence-guided recovery and memory-gated regression prevention within a single lineage.

Progressive convergence (seed 3). Seed 3 improves from approximately -20 to 148 after an L2 refactor replaces an absolute-distance penalty with potential-difference approach shaping. The evidence organizer had reported that the absolute-distance term provided an uninformative gradient—constant penalty regardless of approach direction. An overly restrictive adjustment then causes a temporary regression. The intervention memory enables cross-round comparison: the pre-regression version achieved 148 with a specific contact-semantic formulation, localizing the regression to the most recent change. The subsequent L2 edit reconstructs the contact semantic as a reachable gradient from single-leg contact toward simultaneous stable contact; the next policy reaches 254. These revisions remain within a consistent signal framework, and the hierarchical editing constraint prevents simultaneous modification of the approach shaping and the contact semantic—an entanglement that would have made attribution impossible.

Across both trajectories, component evidence identifies a principal semantic failure; the editor applies a targeted revision; and intervention memory preserves the best solution and enables cross-round comparison.

D. Cross-Task Validation

BipedalWalker-v3 validates CREATE on continuous-action locomotion as a secondary domain. All five lineages cross the threshold within three evaluations (Table II); selected policies obtain 310.82 ± 5.03 test fitness. Mechanism ablations are reserved for LunarLander-v3 (Section V-B), where component-level attribution is the central question.

In summary, structured evidence, bounded editing, and intervention memory form a coupled diagnostic architecture: evidence without editing provides diagnosis without attribution; editing without evidence constrains action without guiding it.

VI. CONCLUSION

This study presented CREATE, a closed-loop reward editing agent that converts failed policy training into diagnostic evidence for reliable reward repair. Through a detailed case study on LunarLander-v3, CREATE solved all five lineages under a ten-evaluation budget, whereas budget-matched independent generation and stateless iterative revision both solved zero. Mechanism ablations demonstrated that enriched evidence and hierarchical semantic editing are coupled requirements: enriched evidence enables the editor to distinguish mechanistically distinct failure modes, while bounded editing ensures each retraining run serves as a controlled attribution test. Intervention memory preserves the evidence–action–outcome chain across rounds and prevents regression. Cross-task validation on BipedalWalker-v3 provided supporting evidence of transfer to continuous-action locomotion. While these results establish the diagnostic closed-loop architecture as an effective paradigm for reward repair, the evidence-enrichment design was developed and ablated exclusively on LunarLander; its transfer to substantially different state-action structures remains an open question. Future work should examine evidence-organizer design for broader domain coverage, robustness to training-instability noise, and integration with stronger policy optimization methods.¹

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning*, 1999, pp. 278–287.
- [3] D. Hadfield-Menell, S. Milli, P. Abbeel, S. Russell, and A. Dragan, "Inverse reward design," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.
- [5] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, "Language to rewards for robotic skill synthesis," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 374–404.
- [6] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, "Text2reward: Reward shaping with language models for reinforcement learning," in *International Conference on Learning Representations*, 2024.
- [7] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations*, 2024.
- [8] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, and X. Li, "A large language model-driven reward design framework via dynamic feedback for reinforcement learning," *Knowledge-Based Systems*, vol. 326, p. 114065, 2025.
- [9] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires, "REvolve: Reward evolution with large language models using human feedback," in *International Conference on Learning Representations*, 2025.
- [10] X. Zhu, J. Du, Q. Fu, and L. Chen, "LLM-based reward engineering for reinforcement learning: A chain of thought approach," in *2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, 2025.

- [11] Y. Wang, Y. Tang, B. Liu, X. Liu, and D. Shang, "When LLM reward design fails: Diagnostic-driven refinement for sparse structured RL," *arXiv preprint arXiv:2605.28918*, 2026.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.
- [13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhmoey, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [15] N. Gao, X. Zhang, X. Jiang, M. You, M. Zhang, and Y. Deng, "RF-Agent: Automated reward function design via language agent tree search," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [16] Z. Wu, J. Lu, Y. Chen, Y. Liu, Y. Zhuang, and L. Hu, "STRIDE: Automating reward design, deep reinforcement learning training and feedback optimization in humanoid robotics locomotion," *arXiv preprint arXiv:2502.04692*, 2025.

¹Code and supplementary material: <https://github.com/Nicole-ying/expert-reward-agent>